A Configurable CGRA Mesh on FPGA: Design and Pre-Silicon Evidence

Mattia Fait
Advanced Logic Design
University of Trento
Trento, Italy

***Abstract*—This report presents a UART-controlled coarse-grained reconfigurable array (CGRA) with a default 4×4 mesh and rectangular geometries of up to sixteen 16-bit processing elements, each holding a single result register. A host configures operations, supplies edge operands and requests a count of logical steps. We map element-wise operations, local multiply–accumulate reductions, matrix–vector products with stationary coefficients and host reduction, convolution and prefix scans, and expose them through a C API and a declarative command-line front end. Two protocols are specified: a checksummed command protocol with a fused execute transaction, and an experimental framed extension with CRC, session sequencing, exact-replay deduplication and on-chip resident vectors. Evidence is layered and kept separate: independent scalar references and C/RTL byte-exact replay over eight geometries; formal proofs of selected safety properties of the production VHDL; and real Vivado 2026.1 implementation runs, in which the default mesh closes routed timing at 10 ns and a passing candidate at 5.859 ns (170.7 MHz) was found. Fusion divides transaction counts by 2.0–4.0 on five workloads, at up to 9% more bytes, and a first-order link model shows it to be faster always for two workloads and, for the other three, whenever the per-transaction turnaround exceeds 58–174 μs. Resident vectors reduce eight repeated 256-element dot products from 49 976 to 1 856 bytes. A roofline-style bound places every kernel about four orders of magnitude below the array's compute ceiling: the serial link, not the fabric, limits throughput. These are counts and models, not elapsed times: board operation, link latency and end-to-end speedup remain unmeasured.**

***Index Terms*—CGRA, reconfigurable computing, systolic array, FPGA, dataflow mapping, UART protocol, formal verification, static timing analysis.**

I. Introduction

A coarse-grained reconfigurable array (CGRA) selects word-level operations and communication paths at runtime [1]. This project implements such a fabric in VHDL and drives it from a host through a UART interface. The default geometry is 4×4; the integrated controller, protocol and host support positive $R \times C$ meshes with $RC \leq 16$ and 16-bit data. Reconfiguration changes the program, not the physical mesh dimensions.

The project makes five connected contributions:

- a step-controlled mesh with one result register per processing element (PE) and north/west edge injection (Section III);
- a versioned command protocol with a fused execute transaction and an explicit contract for detected transfer failures (Section IV) and an experimental framed extension with CRC, sessions, exact-replay deduplication and resident vectors (Section VIII);
- mappings of vector, reduction, matrix, convolution and scan kernels, with an explicit account of computing versus forwarding PEs and of host reduction (Section V);
- an installed C kernel API, a byte-accurate emulator and a declarative command-line tool (Section VI);
- a layered body of pre-silicon evidence: differential simulation, formal proofs on production RTL, and real Vivado implementation runs (Sections IX–X).

Throughout, the report separates what was *executed* from what is *inferred*. Table I summarises the evidence per layer. Wire-cost results are byte and transaction counts, not elapsed times; routed static timing is a constraint check, not a board measurement; a reached cover is reachability, not proof. Physical board operation has not been tested and no CPU speedup is claimed.

Report revision of 15 October 2026. Every quantitative claim is traceable to a dated record under `doc/verification/` containing the commands, tool versions, exit statuses and source hashes of the run. No FPGA board was available: all results are pre-silicon.

II. Background and Related Work

CGRAs configure word-level operations and communication rather than individual logic gates [1]–[3]. Published designs explore different balances: ADRES couples a VLIW processor to a reconfigurable matrix [4]; MorphoSys provides richer communication resources [5]; HyCUBE supports reconfigurable multi-hop paths [6]; PipeRench virtualises pipelined computations [7]; and DySER integrates specialised datapaths with a processor [8]. Plasticine organises compute and memory around parallel patterns [9], while CGRA-ME supplies a framework for architecture and mapping exploration [10]. The present design uses a smaller resource set: nearest-neighbour links, one result register per PE and one configuration context at a time.

A. Systolic computation and stationary data

Systolic computation coordinates local operations and communication over successive steps [11], [12]; modern matrix engines such as the TPU use this principle at a much larger scale [13]. A stationary-data label describes which value remains local while others move. In an *output-stationary* mapping, an output's partial sum stays in local storage as its products accumulate. In a *weight-stationary* mapping,

TABLE I

EVIDENCE MAP: WHAT WAS EXECUTED FOR EACH LAYER OF THE SYSTEM. COUNTS ARE NUMBERS OF CHECKS OR EVENTS, NOT COVERAGE PERCENTAGES. EACH ENTRY REFERS TO A DATED RECORD UNDER `doc/verification/`.

Layer	Simulation and replay	Formal, production RTL	Vivado synthesis and routed STA	FPGA board
PE datapath, ALU	● 1 024 C/RTL differential results; every opcode and selector pair	● all 16 opcodes, both operand muxes, symbolic 16-bit inputs	● part of the routed 4×4 design	○ not run
Mesh wiring	● 8 device shapes end to end; standalone arrays up to 8×8	● 8 enumerated shapes, composed with the PE proof	● five 16-PE shapes close at 10 ns	○ not run
Controller, v2/v3 protocol	● 504 model-generated transactions replayed byte-exactly, $D = 1$–4	◐ safety PROVED on 9 instances; completion only conditional	● 4×4 bitstream gate; best candidate 5.859 ns	○ not run
UART TX/RX	● 12 phase/period cases, $\pm 2\%$ rate, glitch, break	● digital sampling model at divisors 3 and 868	● two recognised synchronisers (CDC)	○ no physical link
Resident-block extension	● 886 transcript events $\times$ $D = 1$–4; reset/concurrency faults	◐ receiver, RAM, sequencer PROVED; final consolidated run pending	● one routed 4×4 instance at 10 ns	○ not run
Host library and CLI	● 3 519 checks, release and ASan/UBSan; PTY transport faults	– no formal claim	–	○ no serial bridge

● executed and passed ◐ passed within a stated bound or condition ○ not executed – not applicable

weights remain local for reuse [14]. Eyeriss's row-stationary organisation exploits several kinds of reuse [15]. These labels describe data movement; they do not by themselves specify a particular number of registers or an interconnect.

Our MAC instruction already supports local output accumulation. The limiting resource is independent forwarding: a PE exposes only its result register to neighbours. Section V explains why the implemented matrix mapping uses stationary vector coefficients, product delay lines and host reduction. It does not establish that every output-stationary schedule is impossible on the mesh.

B. Scope of the project

The contribution is a reproducible teaching system connecting configurable RTL, a serial protocol, a C API and a declarative front end. Functional simulation, independent scalar kernel references and measured host byte counts provide different forms of evidence. Neither emulator execution nor RTL simulation constitutes a measurement on an FPGA board.

III. FABRIC ARCHITECTURE

The fabric is an $R \times C$ mesh (default 4×4) of identical processing elements operating on 16-bit signed words. The complete device supports positive dimensions with $RC \leq 16$. All PEs share a single clock domain and advance synchronously; a global *step* enable, asserted by the controller, gates every PE register so that the host advances the whole array a deterministic number of cycles per command.

A. Processing element

Figure 1 shows the PE datapath. Each PE has two operand multiplexers that independently select one of seven sources: the registered output of a north, south, east or west neighbour; a 16-bit immediate carried in the configuration word; the PE's own register (a feedback path); or a constant zero. The selected

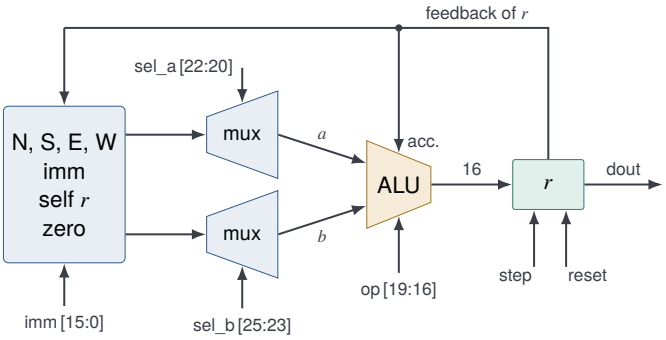


Fig. 1. Processing-element datapath. Two 8-way multiplexers select each operand from the four neighbours, the immediate, the PE's own register or zero; configuration fields are shown with their bit ranges. MAC and ACC also read r directly. The register captures on *step* and is cleared by reset.

operands drive an arithmetic-logic unit; its result is latched into the single output register when *step* is asserted.

The ALU implements sixteen operations selected by a 4-bit opcode: a no-op that holds the register; a pass-through; addition, subtraction and multiplication (the low 16 bits of the product); a multiply-accumulate and a simple accumulate, both folding into the register; the bitwise AND/OR /XOR; a logical left shift and an arithmetic right shift, whose count is the low four bits of the second operand; signed maximum, minimum and absolute value; and a constant load. Multiplication and multiply-accumulate keep the low half of the result, so arithmetic wraps modulo 2^{16}, consistent with a 16-bit datapath; in particular $|-32768|$ wraps to -32768.

A single 32-bit configuration word fully specifies a PE: bits 15–0 hold the immediate, bits 19–16 the opcode, and bits 22–20 and 25–23 the two operand selects; bits 31–26 are reserved and ignored. The eighth select code is reserved and yields zero. One word per PE constitutes a complete array configuration.

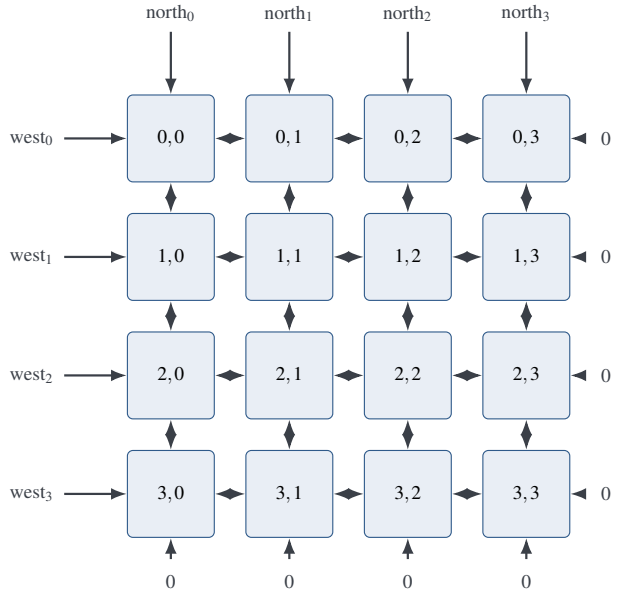


Fig. 2. The 4×4 mesh. Solid arrows are nearest-neighbour register connections; operands are injected only at the north edge (per column) and the west edge (per row), while the south and east edges read zero. This edge-only injection is the central constraint on dataflow mapping (Section V).

B. Mesh interconnect and operand injection

Figure 2 shows the interconnect. Internally, each PE reads the *registered* outputs of its four neighbours, so data advances by at most one hop per step. External operands enter only at two edges: the north input port feeds row 0 (one value per column) and the west input port feeds column 0 (one value per row). The south and east edges read a constant zero. All RC PE registers are exposed for read-back.

Streamed operands reach interior PEs through their neighbours. Configuration can independently load an immediate for every PE, but repeated configuration has its own wire cost. Edge injection and one exposed result register per PE therefore constrain communication schedules, rather than forbidding all alternative ways to supply operands.

IV. Control and Protocol

The host drives the fabric through a byte-oriented UART protocol interpreted by a control finite-state machine (FSM). The protocol is deliberately minimal: single-byte commands with lengths determined by geometry and the tap mask, so that the same message stream can be produced and checked by both the hardware and a software model.

A. Command set

Table II lists the commands. Configuration and input transfers carry an 8-bit additive checksum; a mismatch produces a negative acknowledgement. Register read-back also carries a checksum. These checks do not detect every possible corruption. The identification reply reports the array geometry, so the host can adapt at run time rather than assuming fixed dimensions.

TABLE II
UART command set. `EXEC` is the version-3 addition analysed in Section IV-C; the rest are version 2.

Cmd	Payload	Reply
`ID`	—	signature, version, geometry
`CFG`	RC words + ck	ACK / NACK
`WR`	$(R+C)$ words + ck	ACK / NACK
`RUN`	step count N	ACK (after N steps)
`RD`	—	RC registers + ck
`RST`	—	ACK (clears registers)
`EXEC`	N, flags, tap mask, $(R+C)$ words + ck	masked registers + ck + status

The full device supports positive R and C with $RC \leq 16$ and a 16-bit datapath. The fused command retains a two-byte tap mask for every geometry. Bits beyond the PE count are ignored by the controller and rejected by the host API. Configuration and read-back use row-major indices $rC+c$; input payloads contain all R west words followed by all C north words.

B. A transaction on the wire

To make the framing concrete, consider writing the edge inputs west $= (1,0,0,0)$ and north $= (5,0,0,0)$ with the `WR` command. The host sends the command byte `0x03` followed by the four west then four north values as 16-bit little-endian words, and finally the additive checksum:

```
03  0100 0000 0000 0000  0500 0000 0000 0000  06
```

The sixteen payload bytes sum to $1+5=6$, so the trailing checksum is `0x06`; the device recomputes the same sum, finds it matches, and replies with a single `ACK` byte (`0x79`). If corruption changes the sum, the device replies `NACK` (`0x1F`); the host may retry under the fault model described below. Both `CFG` and `WR` use this request/reply shape.

Configuration and input words are committed as they arrive, before checksum validation. A `NACK` reports a failed transfer but does not restore the previous words; a complete valid transfer is required before subsequent execution. A rejected `EXEC` similarly latches its inputs but does not reset or advance the PE registers. The differential tests of Section X check this observable error behavior.

C. The fused execute transaction

The six version-2 commands are complete but not economical, and the reason is visible in how a kernel actually uses them. Every element-wise chunk issues the same triple—write the edge operands, run, read the registers—so each chunk costs three round trips, and the read returns all sixteen registers even when the mapping produced four results. Section VII shows this is where protocol overhead enters the cost model; physical latency remains unmeasured.

`EXEC` collapses that triple into one transaction (Figure 3). Its payload is the `WR` operand block preceded by three fields: the step count, a flags byte, and a 16-bit *tap mask* in which bit i selects PE register i. The device latches the operands, optionally clears the datapath (flags bit 0, a folded `RST`), steps

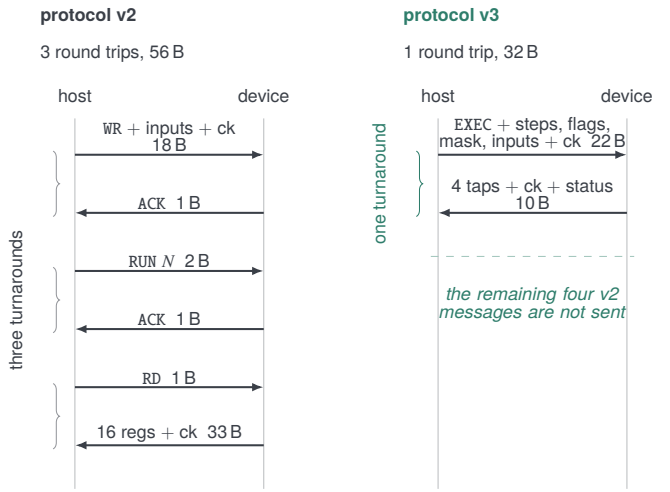


Fig. 3. One element-wise chunk on the wire. The unfused sequence (left) spends three turnarounds and returns all sixteen registers; `EXEC` (right) spends one and returns only the four registers the tap mask names. The optional datapath clear is folded into the flags byte. Reply framing assumes that the command boundary and tap mask arrive intact.

the array, and streams back only the masked registers, a data checksum and a status byte. A four-lane element-wise chunk therefore ships eight result bytes instead of thirty-two, in one turnaround instead of three.

The reply length follows the tap mask received by the controller. If the mask and command boundaries arrive intact, the host knows that length even when another payload byte fails checksum validation: the device returns current registers with `NACK` and does not step the array. A corrupted mask can instead change the reply length, so the checksum does not guarantee framing. The library retries a complete, checksum-valid negative reply only when the fused reset was requested. A timeout, I/O failure or malformed reply stops the session, including when reset was requested: idempotent computation does not repair an uncertain parser boundary.

Since the identification reply carries the protocol version, a host that meets an older bitstream simply falls back to the unfused sequence. Both paths are held to the same known-answer suite (Section X).

D. Controller state machine

Figure 4 shows the controller of `cgra_ctrl` with the RTL state names. From `S_IDLE` the first received byte selects a branch. Short transactions (`ID`, `RST` and the rejection of an unknown opcode) reply immediately through the shared transmit handshake. Bulk transfers accumulate their payload, receive and verify the checksum in a dedicated state, and reply `ACK` or `NACK`. The `RUN` branch produces N enabled PE captures at the configured cadence before acknowledging; `RD` streams the registers and their checksum.

The `EXEC` path shares existing controller logic. Its header state falls through into the *same* operand-payload state `WR` uses and the *same* step engine `RUN` uses; only the return targets differ, held in two small registers. The fused command is thus a different path through the existing states rather than a parallel implementation of them—which is what keeps one byte-accurate software model able to mirror both.

A watchdog guards every state that waits for a payload or checksum byte: if no byte arrives within `G_TIMEOUT_CYCLES` clocks (10^8 by default, i.e. 1 s at 100 MHz) the machine returns to idle. This does not provide reliable framing under byte insertion, deletion or command corruption. The protocol has no delimiter or sequence number, and its additive checksum also misses compensating payload changes; the framed extension of Section VIII adds both. Retry assumes intact commands and frame lengths; it provides no general exactly-once guarantee.

After an uncertain transfer the library clears cached identity. Further protocol calls return a desynchronization error (`CGRA_ERR_DESYNC`). It sends no implicit `RST`: a controller waiting for a `RUN` argument could interpret that byte as six steps. The caller must close the handle, restore the parser to idle by an out-of-band device reset or its watchdog after pending traffic has ended, and reopen. Configuration, inputs and datapath must then be reinitialized. Reopening a serial port alone does not reset the remote controller.

V. Dataflow Mapping

Let the device geometry be $R \times C$ and $L = \min(R, C)$. A step updates all PEs from the preceding register values. Host edge inputs remain fixed until a subsequent input command changes them.

A. Element-wise operations and reductions

The diagonal mapping injects vector a from the north and b from the west. Cells above the diagonal pass north values southward; those below pass west values eastward. After L steps each of the L diagonal PEs holds one independent result. Immediate operations replace an operand at the computing PE, rather than requiring a second vector transfer. Incomplete chunks are zero-padded and only valid outputs are returned.

For dot product, $\mathrm{PE}(0,0)$ performs $r \leftarrow r + a_i b_i$. The host clears its register once, supplies one pair per step and reads the scalar at the end. The partial sum remains local throughout: this is a concrete example of local output-stationary accumulation supported by the existing ISA.

B. Stationary coefficients with host reduction

For $y = Ax$, the engine tiles A into $R \times C$ blocks. For each column tile, row 0 holds the corresponding coefficients of x in its configuration immediates. Each step injects one matrix row from the north: row 0 multiplies, and rows 1 through $R-1$ pass the preceding products southward. After R steps, the snapshot satisfies

$$q_{R-1-i,c} = A_{i,c} x_c, \qquad y_i \mathrel{+}= \sum_{c=0}^{C-1} q_{R-1-i,c}, \tag{1}$$

with local tile indices and zero padding outside the matrix. The host performs the sum and accumulates across column tiles; it is part of the algorithm.

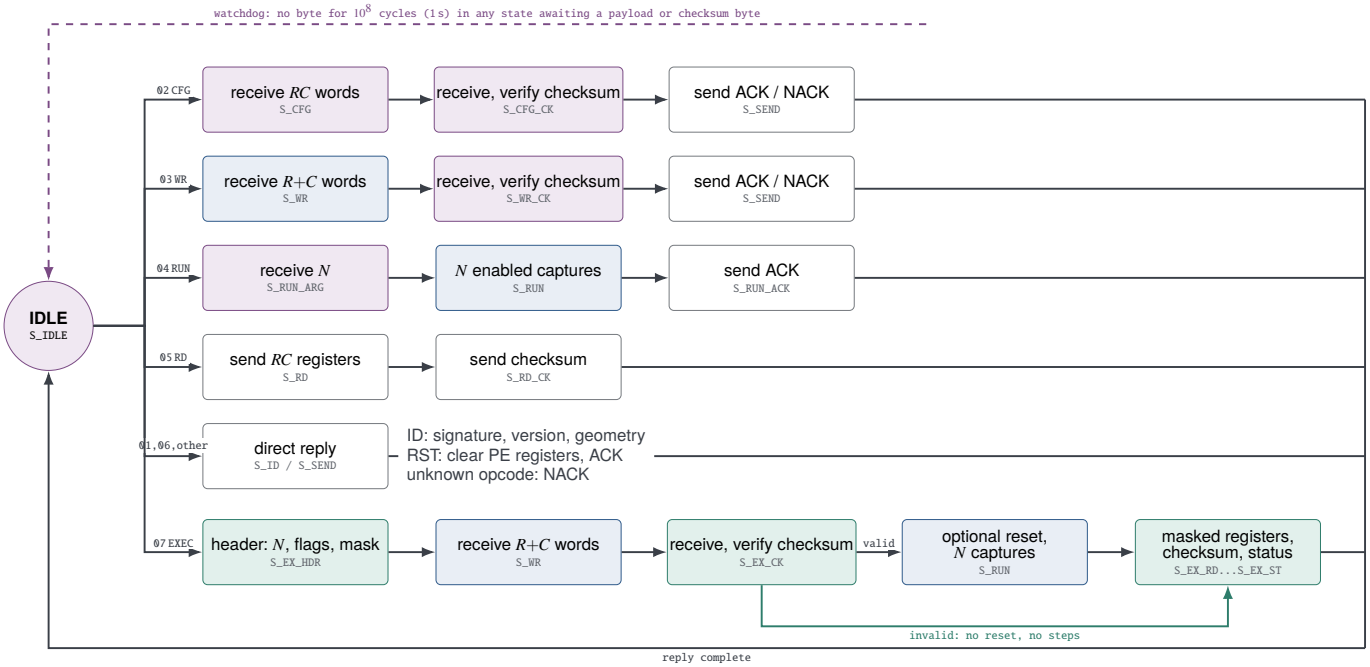


Fig. 4. Controller state machine of `cgra_ctrl`. Boxes give the function and the RTL state(s) in typewriter; byte loops and the shared transmit handshake are folded into the boxes. Blue states are shared by two commands (`WR`/`EXEC` reception, `RUN`/`EXEC` stepping) and return through a stored target; green states exist only for `EXEC`; white states transmit. A rejected `EXEC` checksum skips reset and stepping but still returns the current registers with `NACK`. Every reply ends in `S_IDLE`; the watchdog (dashed) aborts stalled reception.

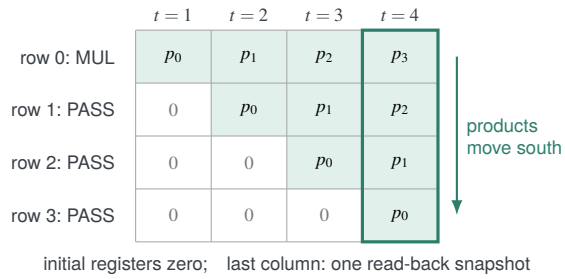


Fig. 5. One column of a 4×4 matrix tile. Here $p_i = A_{i,c}x_c$. Time increases from left to right; the last column is the read-back snapshot. The host sums snapshot PE rows across all columns to obtain output rows in reverse order.

On a full tile, C PEs perform useful multiplications per step and $(R-1)C$ serve as product storage/forwarding. Thus a 4×4 mesh has four active multipliers and twelve delay cells in this mapping, not sixteen useful MACs per step. The steady full-tile multiplier utilisation is $1/R$; padding and skipped tiles change the effective utilisation for a workload. The term weight-stationary refers here specifically to x held in row 0's immediates; the complete computation includes host reduction.

C. Forwarding limits and alternative schedules

Consider an interior PE expected to retain a sum s_t, forward activation a_t east, and forward weight b_t south during the same step. After a MAC, its only exposed register contains $s_{t+1} = s_t + a_t b_t$; both neighbours see that value, not two independently forwarded operands. The intended one-MAC-per PE-per-step schedule therefore needs additional communication or storage. This is a throughput/resource conflict, not an impossibility proof for all output-stationary mappings. Dedicated routing PEs, time multiplexing, reconfiguration between phases, or extra registers could implement other schedules at different area and communication costs.

D. Convolution and prefix scan

Full convolution is expressed as a Toeplitz matrix–vector product [16]. The host generates only tiles intersecting the band: element (i,j) is $h[i-j]$ for $0 \le i-j < k$, and zero otherwise. For an input of length n and an $R\times C$ tile, at most $\lceil n/C \rceil (\lceil (k+C-1)/R \rceil + 1)$ candidate tiles are visited. Each costs $O(RC)$ host work; bounded tile storage replaces the dense $(n+k-1)n$ workspace. Zero-weight and all-zero tiles still skip transfers.

Inclusive scan [17] uses row 0. PE$(0,c)$ combines its west value with the element on north input c; the previous block's last result enters west input 0 as a carry. After C steps all C prefix results have settled. The configuration is independent of the input values and is loaded once per call. Supported folds are ADD, MUL, MIN and MAX, with 16-bit arithmetic.

VI. Host Software Stack

The host side (Figure 6) is layered. A C library owns the transport and protocol: it exposes primitives to identify the device, load a configuration, write the edge inputs, advance the array and read the registers, and it implements the checksum, retransmission after complete negative replies, and session shutdown after uncertain transfers. The library also contains

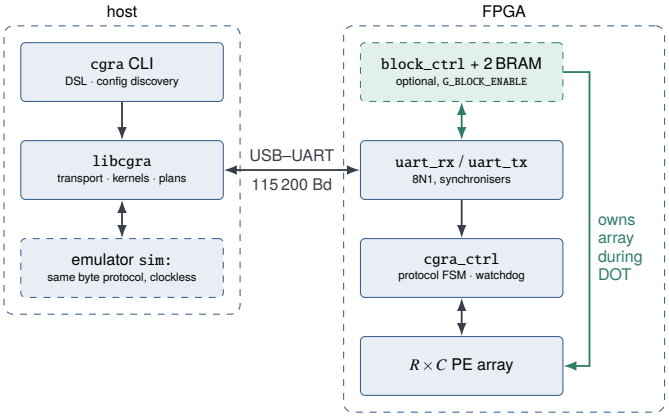


Fig. 6. Host/device architecture. The command-line tool compiles a declarative configuration language into per-PE words and drives the library, which reaches either the FPGA over a USB–UART bridge or the clockless emulator speaking the same byte protocol. The block controller (dashed, green) exists only when the extension is elaborated.

an in-process model of the device that speaks the identical byte-level protocol, so the entire stack runs and is verified without an FPGA; the same model can be exposed on a pseudo-terminal [18] to serve external tools.

The library is also where the fused transaction of Section IV-C is made optional rather than mandatory. It caches the protocol version from a single identification handshake and selects, per chunk, either the fused command or the three-transaction sequence, so one binary drives both a current and an older bitstream. The same switch is what makes the optimisation measurable: because the device model can be told to report the older version and refuse the fused command, identical work can be run down both paths and the resulting transaction and byte counters compared directly (Section X).

The cached identity also carries geometry. Transfers with explicit word counts validate configuration, edge-input and read-back buffer lengths before sending the corresponding command. The original fixed-size API remains a checked 4×4 interface; vector, dot-product, matrix–vector, scan and convolution kernels adapt automatically. The compiler uses $\min(R,C)$ diagonal lanes, the matrix engine uses $R \times C$ tiles and scans use C columns. Explicit placement and output taps are rejected when they do not fit the selected device. The emulator accepts geometries such as `sim:2x3`, allowing the same host binary to exercise rectangular variants without hardware.

Above the library, a command-line tool provides high-level, UNIX-style access. A small declarative configuration language describes named *devices*, *modes* (array configurations, expressed either as a high-level pattern or as explicit per-PE placement), *pipelines* that chain modes, and input/output profiles. Configuration is discovered from standard locations and merged by name, following the drop-in convention used by system daemons, so that a user's library of kernels can be organised across files and overridden selectively. Vectors flow through the tool via standard input and output, allowing kernels to compose in pipelines with ordinary UNIX utilities.

TABLE III
REQUEST AND REPLY SIZES IN BYTES; CHECKSUMS ARE INCLUDED.

Command	TX	RX	Total, 4×4
ID	1	5	6
CFG	$4P+2$	1	67
WR	$2E+2$	1	19
RUN	2	1	3
RD	1	$2P+1$	34
RST	1	1	2
EXEC	$2E+6$	$2t+2$	$24+2t$

The public header and manual define buffer capacities, ownership, errors and aliasing. Handles own their transport; callers retain ownership of data buffers. Vector and scan kernels allow exact in-place operation but reject partial overlap; matrix–vector and convolution require disjoint outputs. Dot product may store its scalar over consumed input. Empty public kernels perform no device I/O. A failure after computation starts can leave partial output; preflight validation does not make an entire kernel transactional.

Calls sharing a handle or the same physical device require caller synchronization. Independent handles do not serialize a common UART target. The command-line front end publishes output only after successful computation, although a final filesystem write can itself fail. Its strict parser validates fields, geometry and pipeline input requirements before executing commands.

VII. Communication and Performance Model

The model separates byte serialization, request/reply count and array steps. Let $P = RC$, $E = R+C$, and let t be the number of selected registers. Counts assume complete successful transactions, no retries and no other port traffic.

The EXEC request adds four bytes to WR: step count, flags and a two-byte tap mask. For a 4×4 diagonal chunk, WR+RUN+RD costs 56 bytes and three transactions; EXEC with four taps costs 32 bytes and one transaction. Untapped cells can contain routing data: the saving comes from omitting registers that the caller does not need, not from assuming they were unwritten.

A. Per-kernel transaction counts

Table IV describes the current public C kernels for nonempty input and a cached, validated identity. Every call configures its mapping; no cross-call configuration cache is assumed. For vector length n, write $K = \lceil n/\min(R,C) \rceil$ and $Q = \lceil n/C \rceil$. For dense $M \times N$ matvec, let $T_R = \lceil M/R \rceil$ and $T_C = \lceil N/C \rceil$, with no skipped tile.

Dot performs CFG, RST, n iterations and a final RD: fusion replaces each WR+RUN pair with EXEC, leaving all three surrounding commands. Scan resets before each block in v2; v3 folds that reset into EXEC. Matvec resets per tile; v3's first EXEC folds in the reset and its last reads the full array. The DSL mode runner has a separate reset contract: a nonempty diagonal/custom call resets initially, adding one RST to the v2 vector expression for default `reset=once`. Public empty

TABLE IV
CURRENT-KERNEL TRANSACTIONS, EXCLUDING INITIAL ID.

Kernel	v2	v3
Vector	$1+3K$	$1+K$
Dot	$3+2n$	$3+n$
Scan	$1+4Q$	$1+Q$
Dense matvec	$T_C[1+(2R+2)T_R]$	$T_C[1+RT_R]$

kernels have host-defined zero/no-op results and issue no commands. An uncached identity adds one transaction and six bytes.

For sparse matvec, if B tile pairs actually execute and C_a column tiles contain at least one executed pair, the counts become $C_a+(2R+2)B$ in v2 and C_a+RB in v3. Zero-weight column tiles and zero matrix tiles can be omitted exactly. The scan for zeros still consumes host work. Convolution uses the same transfer rules with implicit Toeplitz tiles; structurally zero row ranges are skipped without scanning their elements.

B. Serialization, turnaround and the value of fusion

At 115 200 Bd with 8N1 framing, a byte occupies ten bit times,

$$t_b = \frac{10}{115\,200}\,\mathrm{s} \simeq 86.8\,\mu\mathrm{s}, \tag{2}$$

which at a 100 MHz clock is 8 680 clock cycles per byte. A 56-byte unfused chunk therefore needs about 4.86 ms of serialization and a 32-byte fused chunk about 2.78 ms. A first-order model for one call is

$$T \simeq B t_b + J\tau + T_{\mathrm{host}} + T_{\mathrm{device}}, \tag{3}$$

where B is the number of wire bytes, J the number of transactions and τ the additional turnaround per transaction. USB buffering and driver latency timers can delay short replies [19], so τ depends on the bridge, driver and settings and must be measured. The device term is small: a step lasts D/f_{clk}, i.e. 20 ns for $D=2$, against $t_b \approx 87\,\mu$s per byte.

Neglecting the host and device terms, fusion (v3) is faster than the unfused sequence (v2) exactly when

$$\tau > \tau^* = t_b \frac{B_{\mathrm{v3}} - B_{\mathrm{v2}}}{J_{\mathrm{v2}} - J_{\mathrm{v3}}}. \tag{4}$$

For vector addition and scan, fusion reduces both bytes and transactions, so $\tau^* < 0$ and v3 is never slower. For dot product, matrix–vector and convolution it trades extra bytes for fewer transactions; the measured counts of Table VIII give $\tau^* \approx 174\,\mu$s, $58\,\mu$s and $58\,\mu$s respectively. Figure 7 plots the resulting ratio $T_{\mathrm{v3}}/T_{\mathrm{v2}}$. With a typical USB latency timer in the millisecond range the model predicts that fusion pays off for every workload, approaching the transaction ratio of 0.25–0.50; with an ideal link it costs up to 9% for dot product. These are model predictions from measured counts, not elapsed times.

C. A roofline view: the link, not the array, is the bound

A roofline model [20] bounds the attainable rate by the minimum of compute throughput and operational intensity

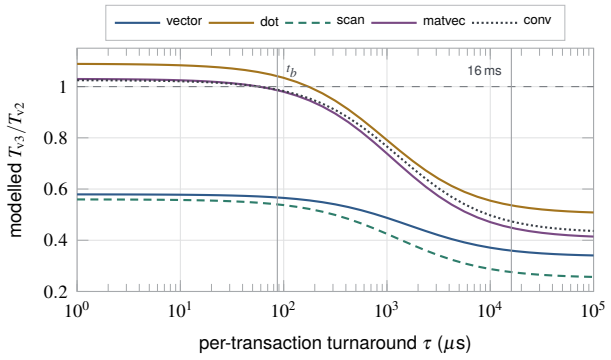


Fig. 7. Modelled call-time ratio of fused (v3) to unfused (v2) execution as a function of the turnaround τ, from the measured byte and transaction counts of Table VIII and $T = Bt_b + J\tau$ at 115 200 Bd. Below the dashed line fusion is faster. Vertical lines mark one byte time t_b and a 16 ms USB latency timer. Model, not measurement.

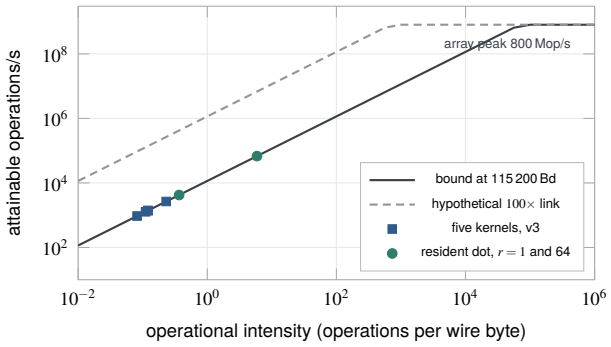


Fig. 8. Roofline-style bound [20]. Intensity is the number of arithmetic operations of the scalar reference per wire byte, from measured counts; the attainable rate is the minimum of the array peak (16 PEs at $f_{\mathrm{clk}}/D = 50$ MHz) and intensity times the link byte rate. All kernels lie about four orders of magnitude below the ridge. Model bound, not a measured rate.

times bandwidth. We take as compute ceiling one operation per PE per step, $16 \cdot f_{\mathrm{clk}}/D = 800$ M operations/s for the default build, and as bandwidth the link byte rate $1/t_b = 11\,520$ B/s. Operational intensity counts the multiplications and additions of the scalar reference (e.g. $2n-1$ for a dot product of length n) per wire byte measured for the call. The ridge point lies at 69 444 operations per byte.

Figure 8 places the five kernels and the resident dot product on this model. All points lie between 0.08 and 6 operations per byte, roughly 10^4 below the ridge: every workload is bound by the link, and the array would remain mostly idle even at full overlap. Two conclusions follow. First, raising f_{clk} or reducing D cannot improve end-to-end throughput on this link within the model; timing closure matters for correctness margins, not for speed. Second, only higher intensity—data reuse on chip, as in the resident extension—or a faster link moves a workload towards the ridge. A 100× faster link (dashed) would still leave all kernels link-bound.

D. Geometry trade-off at sixteen PEs

The five rectangular shapes with sixteen PEs execute the same kernels with different efficiency (Figure 9). In the

mappings of Section V, the diagonal uses $\min(R,C)$ PEs, the matrix–vector and scan mappings use the C PEs of row 0, and the dot product a single PE. The square favours element-wise diagonals; wide shapes favour row-oriented mappings; tall shapes are poor for all of them. Measured wire bytes show the communication side of the same trade-off: input words scale with $R+C$ per transfer and read-back with RC, so the strips pay for long edges on vector and dot traffic, while the 1×16 strip transfers fewest bytes for scan and matrix–vector because each chunk covers sixteen columns. All transaction counts in the figure equal the closed forms of Table IV (including the zero-weight tile skips), which the generating script checks for every shape. Together with the resource sweep of Section IX-D, this indicates that the shape should follow the dominant kernel; no shape is best for all five.

VIII. Experimental Resident-Block Extension

The command protocol of Section IV has two structural limits. Its framing is implicit, so byte insertion or loss can desynchronise the parser, and every computation must stream its operands over the link. A dot product of length n transfers $O(n)$ bytes on *each* call, even when the same vectors are reused. The resident-block extension addresses both limits for one kernel, and is evaluated with the cost model of Section VII. It is compiled in only with the `G_BLOCK_ENABLE` generic (disabled by default), remains experimental, and leaves the legacy protocol unchanged until a host selects it.

A. Selection, framing and sessions

A single legacy-idle preflight byte (`0x08`) asks for the extension. An enabled controller replies with a marker, block revision, chunk size and capacity; an older controller replies `NACK` to that one byte, so probing cannot make old RTL interpret framed payload as legacy commands. A successful preflight latches framed mode until an external device reset; there is deliberately no in-band escape back to legacy decoding that corrupted payload could trigger.

Each frame (Figure 10) is delimited by `0x7E` and byte-stuffed in the style of HDLC [21]: `0x7E` and `0x7D` inside the body become `0x7D` followed by the byte XOR `0x20`. A ten-byte header carries revision, opcode, a 32-bit session nonce, a 16-bit sequence number and the payload length. A CRC-16 with polynomial `0x1021`, initial value `0xFFFF` and no reflection covers the header and the payload [22], [23]. Revision, length, canonical escaping and CRC are all validated before a request is published; malformed frames are discarded without a reply or any committed effect.

One request may be outstanding. A `HELLO` with a fresh nonce starts a session at sequence 1. The immediately preceding sequence number, with an identical opcode, nonce, length and payload, replays the cached response *without* re-executing it; the same number with different contents returns `CONFLICT`. A lost reply can therefore be recovered by retransmitting the exact frame, without a duplicated upload or computation, in the manner of duplicate suppression in remote procedure call protocols [24]. This is exactly-once behaviour only within one serialised connection and one cached request: an external reset loses the cache, CRC is detection rather than authentication (end-to-end checks remain the caller's responsibility [25]), and a random nonce reduces but does not exclude reuse. The public C API deliberately offers no automatic retry; it poisons the handle after an uncertain transfer, as in the legacy path.

TABLE V
Post-synthesis cost of the extension. Vivado 2026.1, `xc7a100tcsg324-1`, 4×4, $D=2$, 10 ns, identical sources, tool and strategy (`flatten_hierarchy none`).

Build	LUT	FF	DSP	BRAM tile
Legacy only	5 017	1 034	16	0
With block path	8 459	4 348	16	1
Difference	+3 442	+3 314	0	+1

B. Resident dot product and its cost

Two 1024×16-bit banks in block RAM hold vectors a and b. `UPLOAD` appends up to 64 words per frame and commits each frame atomically after complete validation; `DOT` n computes $\sum_{i<n} a_i b_i$ modulo 2^{16}; `RESULT` returns the value and a 32-bit execution counter. During `DOT` a hardware sequencer owns the existing array: PE$(0,0)$ is configured as MAC on its edge operands, the array is reset once and exactly n captures follow. Synchronous RAM outputs first enter explicit operand registers, and a dedicated ownership register selects between host and sequencer operands. Their paths into the PEs use the same D-cycle exception as Section IX-A, justified by formally proved guard intervals; RAM-to-register, enable and reset paths stay single-cycle. The current implementation spends $D+3$ clocks per element.

The trade-off is explicit. Only one PE computes, so array utilisation is lower than in the streaming mappings of Section V; in exchange, vectors cross the link once. Figure 11 reports the public-API traffic measured on the emulator for n up to 1 024 and up to 64 repeated calls, each result checked against a scalar reference. Even a single call is cheaper (1 394 against 6 247 bytes at $n=256$): the legacy dot product streams one operand pair per step but transfers a complete edge vector of $R+C$ words each time, whereas an upload carries only the two operands. Each further call costs about 66.1 bytes and two transactions, independent of n. For eight calls at $n=256$ this gives 1 856 bytes in 26 transactions against 49 976 bytes in 2 072 transactions for the fused legacy kernel; the transaction counts follow $2+2\lceil n/64\rceil+2r$ exactly, while byte counts vary slightly with the data because of byte stuffing. On the roofline of Section VII-C this reuse raises the intensity by almost two orders of magnitude, yet the workload remains link-bound.

The area cost is substantial (Table V): enabling the extension adds about 3.4k LUTs, 3.3k flip-flops and one block-RAM tile, chiefly for whole-frame buffering, CRC and deduplication state. The first version prioritises validated framing over area.

IX. Implementation Flow and Timing Requirements

The batch Vivado flows target two Artix-7 devices [26]: the Nexys 4 DDR (`xc7a100tcsg324-1`, default) and the Basys 3

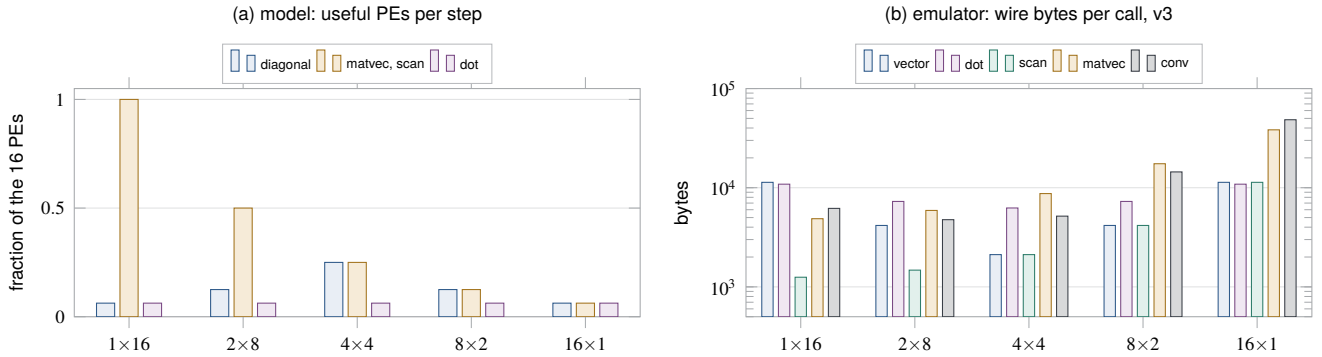


Fig. 9. Geometry trade-off for 16 PEs. (a) Steady-state fraction of PEs performing useful arithmetic per step on a full tile (model): diagonal $\min(R,C)/RC$, matrix–vector and scan C/RC, dot $1/RC$. (b) Measured wire bytes of one public-API call with protocol v3 on the emulator, with the problem sizes of Table VIII; here the vector operand has zeros, so single-column tiles with zero weight are skipped on 16×1. Wide shapes favour the row-oriented mappings; no single shape minimises traffic for every kernel.

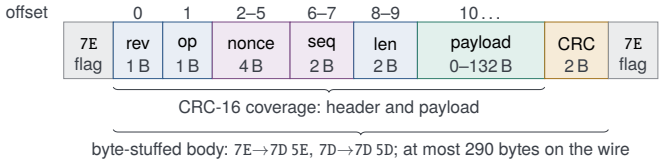


Fig. 10. One resident-block frame. The ten-byte header identifies the session (nonce) and request (sequence); the CRC is computed before stuffing. A reply echoes the opcode with bit 7 set and begins its payload with a status byte. Field widths are schematic.

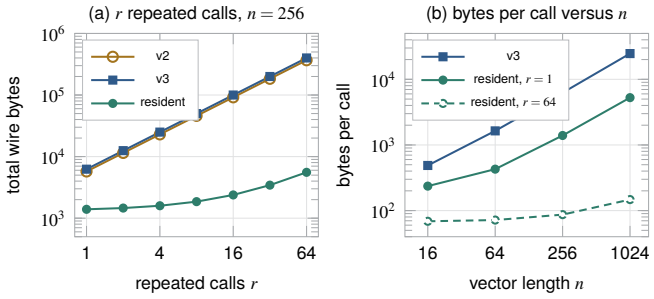


Fig. 11. Repeated dot products through the public API, 4×4 emulator, every result checked against a scalar reference. (a) Legacy traffic grows linearly with the number of calls r; resident traffic is dominated by the one-time upload. (b) Per-call bytes; for $r=64$ the upload is amortised. Counts, not times.

(`xc7a35tcpg236-1`). All recorded implementation runs use Vivado 2026.1 on the default part. The design has one 100 MHz clock domain; asynchronous button and UART inputs enter through two-stage synchronisers [27]. Frequencies recorded in early development notes predate the current timing fixes and are not used in this report.

A. Enabled datapath and multicycle constraints

A PE result depends combinationally on operand selection, arithmetic and the opcode mux. The controller spaces enabled PE captures by $D=$ `G_STEP_DIV` clocks (default 2). Figure 12 shows both an initial reset and subsequent captures. A PE samples *step* on a rising clock edge; that edge captures the current ALU result and launches new register-to-register paths. The step signal is not a separate clock.

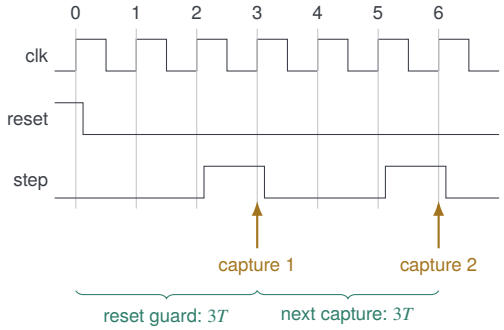


Fig. 12. Example with $D=3$ after EXEC reset. Reset changes the PE registers at edge 0; the first step captures at edge 3, then at edge 6. Stable source registers and the initial guard are required as well as periodic spacing.

The shared constraint procedure applies a D-cycle setup exception and its corresponding $D-1$ hold adjustment to selected datapath paths, following the same-clock multicycle convention [28], [29]. The RTL generic and the exception count come from the same STEP_DIV setting; the clock period is a separate input. GHDL checks spacing after reset, CFG/WR changes and successive steps for $D=1,2,3,4$. This verifies enable sequencing, not netlist selector coverage or actual setup/hold closure. Reset/control paths outside the exception retain their ordinary timing requirements.

B. Timing gates and period search

The Tcl flow checks that expected register/port selections and setup/hold path collections are nonempty. Bitstream generation requires finite, nonnegative setup and hold slack after implementation. Offline tests exercise success and failure branches with synthetic tool replies; the recorded Vivado runs of Section IX-D also pass these gates on actual netlists. Implementation directives are choices to evaluate, not guarantees of improvement [30].

The frequency search synthesises once, reopens that checkpoint for candidate periods and performs placement/routing. The chosen candidate is then rerun with the final implementation recipe, including physical optimisation, and must pass both timing checks again. Only then is $1000/T$ MHz reported for a constrained period T in ns, with a row appended to the validated-history CSV. Placement heuristics need not be monotonic in T, so this establishes a passing candidate rather than a proof of a global maximum. It also does not justify deriving frequency from $T - \mathrm{WNS}$ on a multicycle path.

Changing `create_clock` changes a constraint, not the board oscillator. Operating at another clock needs an actual clock source or clock-management configuration and consistent UART divisors. Even after STA, a physical UART and board acceptance run remain necessary. For sustained computation, f_{clk}/D is the step-rate ceiling before protocol overhead.

C. Implemented changes and comparison limits

The MAC expression adds the accumulator to the full-width product before truncating. This preserves the low 16 bits and exposes a multiply-add form compatible with DSP48E1 arithmetic [31]; it does not prove that Vivado uses the DSP post-adder or that timing improves. The registered read-back mux separates PE selection from later checksum logic. Masked skips include a settling state, and protocol replay verifies the resulting bytes. The recorded run evaluates the combined current design; isolating the resource and timing effects of either change requires matched baselines.

An optional floorplan script selects a DSP-rich region. Historical notes report a regression in one experiment, but lack sufficient provenance for a quantitative comparison here. A new comparison must hold part, source, generics, period and implementation recipe fixed and archive timing and utilisation reports for both cases. EXEC changes controller logic; unchanged PE arithmetic therefore does not imply unchanged total fabric area.

D. Recorded Vivado campaign

Every run below was executed with the real Vivado 2026.1 command-line tool and archived with its inputs, commands, logs, reports, source hashes and a manifest; a failing run is kept as such, not replaced. Unless stated, the configuration is 4×4, $D=2$, a 10 ns constraint, default directives and no floorplan.

Bitstream gate. The complete flow passed all nine gates (synthesis messages, setup and hold, constraint and endpoint coverage, CDC, DRC) and generated a bitstream: setup WNS $+1.862$ ns, hold WHS $+0.098$ ns, 4 971 LUTs, 1 034 flip-flops, 16 DSP48E1 slices and no block RAM. The multicycle exception covers exactly 800 launch registers and 256 PE data endpoints at a 20 ns setup requirement, while all 512 PE clock-enable and reset endpoints keep the 10 ns requirement. No device was programmed.

Geometry sweep. Five 16-PE shapes were implemented from identical sources (Table VI, Figure 13). All close timing at 10 ns. Equal PE counts do not imply equal resources: edge storage and controller logic scale with $R+C$, so the strip shapes use about 14% more flip-flops. Slack is not monotonic in the aspect ratio; a single placement per point does not separate geometry from placer variation.

TABLE VI
MATCHED GEOMETRY SWEEP: ONE ROUTED RUN PER SHAPE, IDENTICAL SOURCES AND TOOL, 10 NS, $D=2$. ALL RUNS USE 16 DSP48E1 SLICES AND NO BLOCK RAM.

Shape	Synth LUT	Routed LUT	FF	WNS/ns	WHS/ns
1×16	5 017	4 999	1 179	+2.433	+0.081
2×8	5 003	4 974	1 066	+0.775	+0.105
4×4	4 999	4 971	1 034	+1.862	+0.098
8×2	5 002	4 975	1 066	+2.350	+0.130
16×1	5 010	4 993	1 179	+1.247	+0.098

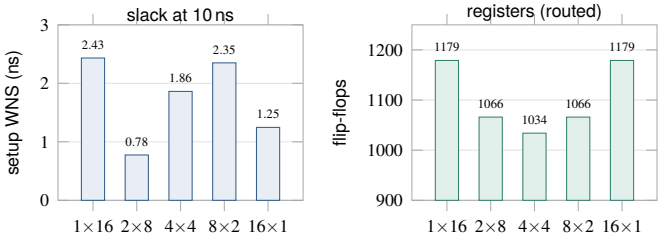


Fig. 13. Five 16-PE geometries implemented from identical sources. All meet the 10 ns constraint (positive slack, left). Register count follows the edge length $R+C$ (right; axis truncated at 900). Each point is one placement, so slack differences include placer variation.

Period search. The search of Section IX-B found a passing candidate at $T = 5.859$ ns, i.e. $1000/T = 170.7$ MHz, with WNS $+0.177$ ns and WHS $+0.122$ ns after the final implementation rerun. With $D=2$ this bounds the sustained step rate at about 85 M steps/s before protocol overhead. It is a constraint result, not a change of the board's 100 MHz oscillator. A first attempt failed its final gate because the requested period (5.859375 ns) differed from the period applied by the tool (5.859 ns); the runner now quantises the period to 1 ps and checks the applied clock. The failed run is retained.

Block extension. The enabled resident-block build closes at 10 ns with WNS $+0.427$ ns and WHS $+0.060$ ns (8 164 LUTs, 4 346 flip-flops, 16 DSPs, one block-RAM tile) after two retained failures at -3.580 and -2.867 ns motivated the explicit operand and ownership registers of Section VIII.

These results establish closure for specific constrained instances. They do not establish a global maximum frequency, power, other parts, or board operation.

E. Planned board acceptance

No board was available for this revision. The acceptance procedure is nevertheless fixed in advance, so that a later run tests stated criteria rather than selecting them after the fact:

1) *Artifact.* Program only a bitstream whose run manifest passes `make check-artifact`, i.e. whose sources, constraints and gate reports match the recorded hashes.
2) *Link.* `cgra probe` must find the device and receive an identification reply whose version and geometry equal the elaborated generics, then pass its known-answer tests.

3) *Function.* The public kernels of Section X must reproduce the scalar references on the board for both protocol versions, including partial chunks and wraparound.
4) *Timing.* Measure the per-byte time and the per-transaction turnaround τ of the actual USB–UART bridge with repeated, warmed-up runs, report distributions rather than single values and state the warm-up, repetition and outlier policy [32]; only then compare against a timed CPU baseline.
5) *Recovery.* Interrupt transfers, assert the reset button and let the watchdog expire; the host must report the session as uncertain and recover only through the documented reset path.

Figure 14 fixes the form in which these results will be reported, with the model predictions already in place for comparison; the measured series are generated from `doc/data/board/` once recorded. Until these steps are executed, every board-level property in this report remains NOT RUN.

X. Verification and Measured Wire Cost

A. Simulation and differential replay

Functional evidence combines independent scalar references, an in-process protocol model, POSIX pseudo-terminal transport tests and VHDL simulation with GHDL [33]. These methods check different boundaries; agreement between the model and RTL alone cannot exclude a shared conceptual error. Scalar references therefore check the numerical kernel results separately.

A protocol bench replays 504 model-generated transactions against the RTL in the style of differential testing [34], comparing every response byte under transmit backpressure at multicycle factors 1–4. It covers zero and 255 steps, sparse/empty masks, checksum rejections and subsequent computation. This exposed an earlier model error: CFG/WR words take effect as they arrive, even if their checksum is rejected. The model now agrees with that RTL behaviour. Watchdog timing is checked separately in RTL because the synchronous model has no clock.

An array differential bench compares 1,024 PE results, covering every opcode and operand-selector pair. Independent wiring tests include seven standalone array shapes through 8×8. These larger standalone arrays do not imply support by the full device protocol. End-to-end host workloads exercise eight supported shapes: 1×1, 1×16, 16×1, 2×3, 3×2, 2×8, 8×2 and 4×4. They cover both protocol versions, signed wraparound, partial vector chunks, rectangular tiles and all four scan operators. Actual host requests captured through a pseudo-terminal are also replayed against the RTL controller and array. Full UART-top replays cover 2×3 (233 transactions) and 3×2 (259 transactions).

The resident-block path has its own replay: C generates 886 frame and reply events, including bit flips, inserted and lost bytes, truncation, resets, watchdog expiry and noise resembling legacy opcodes, and the RTL controller must reproduce every reply for $D = 1$–4; the top-level UART path is replayed for $D = 2$. Each executed `DOT` is checked against an independent signed 64-bit oracle, and exact reset/capture counts distinguish a replayed reply from a repeated computation.

TABLE VII
What each class of evidence establishes, and what it does not.

Method	Establishes	Does not establish
Scalar references	Kernel results on the tested inputs, independently of the model	Correctness on untested inputs
C/RTL replay	Byte-exact agreement of model and RTL on recorded transcripts	Absence of an error shared by both
PTY transport	Host behaviour on a real POSIX terminal with injected faults	Behaviour of a physical USB–UART bridge
Formal proof	Asserted safety properties for all inputs, on enumerated instances	Unasserted properties, other generics, liveness without fairness
Cover	That a stated scenario is reachable within a bound	Any universal property
Routed STA	Setup/hold closure of one netlist under its constraints	Constraint correctness, Fmax optimum, board timing
Wire counters	Bytes and transactions per workload	Elapsed time or speedup

The software suites record 3 519 named checks in the C API, transport, emulator, CLI and benchmark tests, run in release and with AddressSanitizer [35], UndefinedBehaviorSanitizer and LeakSanitizer. In containers with GCC 13 and Clang 18, 3 514 checks pass per profile; five cases needing many file descriptors are skipped explicitly. The GCC static analyser reports no diagnostics. Transport tests inject partial writes, interrupted calls, truncated and malformed replies; an uncertain session must stop without an implicit reset or retry. The count describes the recorded suites, not path coverage.

B. Formal verification of the production RTL

Selected properties are proved on the *production* VHDL rather than on a re-implementation. The GHDL plugin for Yosys [36] imports the PE, array, controller, UART and block sources; properties are written in PSL [37] and bound to the production architectures, observing their internal registers through separate monitors. SymbiYosys runs bounded model checking and k-induction [38]–[40] with the Yices 2 solver [41]. The toolchain (GHDL 6.0.0, Yosys 0.67, SBY 0.61, Yices 2.7.0) is pinned through the repository's Nix flake.

Properties. For the PE, all sixteen opcodes and both operand multiplexers are checked against an independently written unsigned low-half arithmetic specification, with every 16-bit input symbolic. Array wiring is proved on eight enumerated shapes. For the controller, safety invariants cover parser indices, transmit handshakes, the number and spacing of captures for $D = 1$–4, the absence of reset or stepping after a rejected `EXEC`, and the guard between a configuration or input change and the next capture. That guard is the premise of the multicycle exception of Section IX-A: the proof supplies the functional half of the argument, static timing the electrical half.

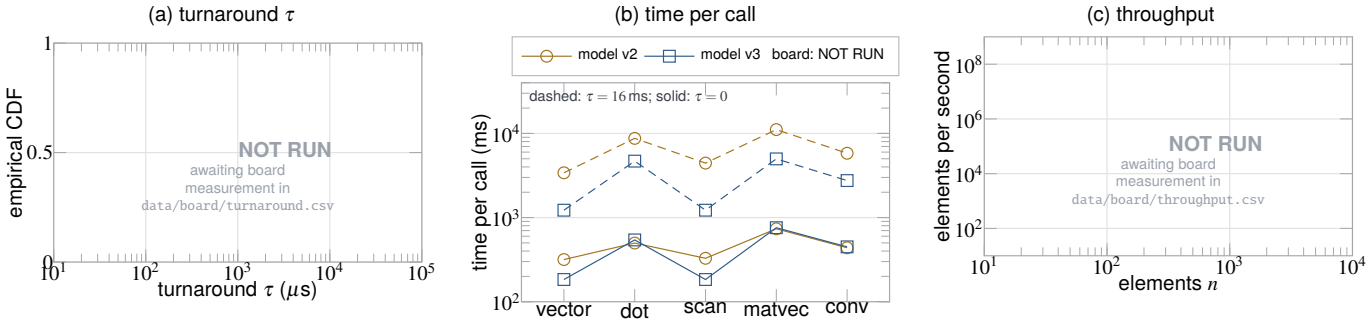


Fig. 14. Board measurements prescribed by the acceptance plan of Section IX-E: (a) empirical distribution of the per-transaction turnaround τ; (b) measured call time against the model of Section VII, drawn for $\tau = 0$ (solid) and $\tau = 16$ ms (dashed) from the counts of Table VIII; (c) throughput against a timed CPU baseline. No board was available: panels (a) and (c) and the measured points of (b) are NOT RUN. The figure is generated from `doc/data/board/` and fills in when measurements are recorded.

Transaction completion is proved only conditionally, by ranking functions under explicit delivery and service obligations; no unconditional liveness claim is made. For the block extension, the receiver (CRC recurrence, escaping, frame acceptance), both RAM banks (retention at an arbitrary address), the `DOT` sequencer and the integrated controller (cache, exact replay without duplicated effects) are covered; the controller's only assumption is an initial reset.

Non-vacuity. The runner counts assertion, assumption and cover cells in the model actually given to the solver and rejects unexpected counts. Following the idea of mutation analysis [42], temporary mutated copies of the design (an adder turned into a subtractor, a dropped step, swapped `EXEC` reply bytes, a north link fed by the PE itself, MSB-first transmission, a bypassed CRC comparison) must each fail with a counterexample, and explicit covers must be reached within their bound.

Results. The last complete full suite on committed sources reports 27 PROVED tasks and 8 required failures (negative and mutation tasks); the cover suite reports 15 BOUNDED_PASS and one PROVED obligation. An extended run with the block properties reports 37 PROVED, 10 required failures and one UNKNOWN (controller, $D = 4$). That instance was then PROVED in a separate run after an inductive invariant was added as an *assertion*, not an assumption; a consolidated final run of the extended suite is still pending. Every UNKNOWN, timeout and frontend error is archived as such. The proofs do not address clock-domain crossing, metastability, placement and routing, or the C software.

C. Reproducible protocol comparison

Table VIII compares the *current* kernels on v2 and v3 of the in-process emulator, using the same default 4×4 geometry and inputs. Each row counts one successful public C API call after identification; CFG and computation are included, the initial ID transaction is excluded, and no retry occurs. The vector and scan operators are ADD. The dense matrix contains only nonzero entries; convolution uses the full-output convention with filter $(1, -2, 3, -4, 5)$ and 64 input samples, producing 68 outputs.

TABLE VIII
CURRENT V2 VERSUS CURRENT V3. TRANSACTIONS AND TOTAL TX+RX BYTES; ID EXCLUDED, ONE CALL PER WORKLOAD. THESE ARE COUNTERS, NOT TIMINGS.

	Transactions		Bytes	
Workload	**v2**	**v3**	**v2**	**v3**
Vector, 256	193	65	3 651	2 115
Dot, 256	515	259	5 735	6 247
Scan, 256	257	65	3 779	2 115
Matvec, 32×32	648	264	8 472	8 728
Conv, $k = 5, n = 64$	336	144	5 040	5 168

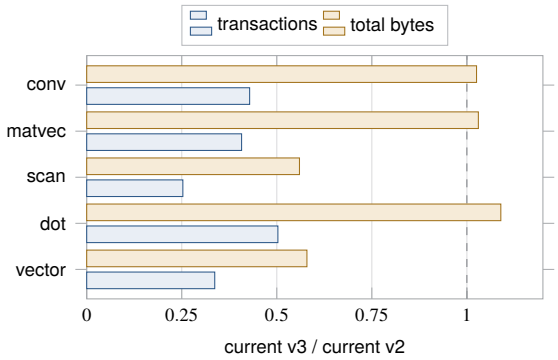


Fig. 15. Same workloads and source data as Table VIII. Ratios below one indicate reduced cost; the dashed line marks parity.

The runner checks each result against an independent scalar implementation and checks an output canary before emitting counters. The CSV, compiler, base commit and hashes of the measured sources are stored under `doc/data`. Transaction formulae in Section VII are checked by the recording script. The table and chart are generated from that CSV; `make -C doc -f doc.mk measure` explicitly regenerates the experiment. An ordinary PDF build consumes the archived data without changing it.

Fusion reduces transactions for all five workloads, but dot product, dense matrix–vector and convolution transmit slightly more total bytes. For these cases the larger EXEC request outweighs reply savings. Fewer turnarounds compensate for

those bytes only when the per-transaction turnaround exceeds the break-even values of Eq. (4) (Section VII-B); wire counters alone do not determine elapsed time.

Both versions already reuse scan configuration and skip zero matrix tiles; this experiment does not isolate those optimisations. Historical before/after figures without a source revision and matching raw outputs cannot establish their speedup and are excluded from this comparison. A physical evaluation must add link settings, board clock, warm-up/repetition policy and a timed CPU baseline. The emulator has no serial-time model, so its wall clock is not an estimate of hardware acceleration.

XI. Conclusion

The project connects a configurable 16-bit mesh, a UART controller, a reusable host API and a declarative front end. The mapping analysis separates the roles a PE can play—diagonal computation, local MAC accumulation, product forwarding—and makes host reduction part of the algorithm. A single result register limits simultaneous independent forwarding but does not forbid local output-stationary accumulation.

Two protocol designs were evaluated with the same counters. Fusing the write–run–read triple divides transaction counts by two to four at a small byte cost; keeping vectors resident on chip removes most of the traffic for repeated reductions, at a substantial area cost and with only one active PE. The link model turns these counts into testable predictions: fusion wins always for vector and scan, and for the other kernels once the turnaround exceeds 58–174 μs, and every kernel is bound by the serial link rather than by the array, so the throughput lever is on-chip reuse or a faster link, not a higher clock. The geometry analysis shows that no 16-PE shape is best for every kernel. Elapsed times remain to be measured; Figure 14 fixes how they will be reported against the model.

The pre-silicon evidence is layered and deliberately kept distinct. Byte-exact C/RTL replay over eight geometries, formal safety proofs on the production VHDL, and real Vivado runs that close timing at 10 ns for five geometries, with a passing candidate at 5.859 ns for the default mesh, each answer a different question; none of them is a board measurement.

Future work follows directly from the stated limits: execute the board acceptance plan of Section IX-E; complete the consolidated formal run and a systematic corruption matrix for the legacy protocol; reduce the area of the framed path; and compare routing or storage extensions of the PE on matched workloads, recording both host and device costs against the same independent references.

References

[1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, no. 6, pp. 118:1–118:39, 2019. [Online]. Available: https://doi.org/10.1145/3357375

[2] R. Hartenstein, "A decade of reconfigurable computing: A visionary retrospective," in *Design, Automation and Test in Europe (DATE)*, 2001, pp. 642–649. [Online]. Available: https://doi.org/10.1109/DATE.2001.915091

[3] A. Podobas, K. Sano, and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE Access*, vol. 8, pp. 146 719–146 743, 2020. [Online]. Available: https://doi.org/10.1109/ACCESS.2020.3012084

[4] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in *Field Programmable Logic and Application (FPL)*, ser. LNCS, vol. 2778. Springer, 2003, pp. 61–70. [Online]. Available: https://doi.org/10.1007/978-3-540-45234-8_7

[5] H. Singh, M.-H. Lee, G. Lu, F. J. Kurdahi, N. Bagherzadeh, and E. M. Chaves Filho, "MorphoSys: An integrated reconfigurable system for data-parallel and computation-intensive applications," *IEEE Transactions on Computers*, vol. 49, no. 5, pp. 465–481, 2000. [Online]. Available: https://doi.org/10.1109/12.859540

[6] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in *54th Annual Design Automation Conference (DAC)*, 2017, pp. 45:1–45:6. [Online]. Available: https://doi.org/10.1145/3061639.3062262

[7] S. C. Goldstein, H. Schmit, M. Moe, M. Budiu, S. Cadambi, R. R. Taylor, and R. Laufer, "PipeRench: A coprocessor for streaming multimedia acceleration," in *26th International Symposium on Computer Architecture (ISCA)*, 1999, pp. 28–39. [Online]. Available: https://doi.org/10.1109/ISCA.1999.765937

[8] V. Govindaraju, C.-H. Ho, and K. Sankaralingam, "Dynamically specialized datapaths for energy efficient computing," in *17th International Symposium on High Performance Computer Architecture (HPCA)*, 2011, pp. 503–514. [Online]. Available: https://doi.org/10.1109/HPCA.2011.5749755

[9] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 389–402. [Online]. Available: https://doi.org/10.1145/3079856.3080256

[10] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson, "CGRA-ME: A unified framework for CGRA modelling and exploration," in *28th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*, 2017, pp. 184–189. [Online]. Available: https://doi.org/10.1109/ASAP.2017.7995277

[11] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in *Sparse Matrix Proceedings 1978*, I. S. Duff and G. W. Stewart, Eds. SIAM, 1979, pp. 256–282.

[12] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982. [Online]. Available: https://doi.org/10.1109/MC.1982.1653825

[13] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3079856.3080246

[14] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017. [Online]. Available: https://doi.org/10.1109/JPROC.2017.2761740

[15] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. [Online]. Available: https://doi.org/10.1109/JSSC.2016.2616357

[16] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013. [Online]. Available: https://www.press.jhu.edu/books/title/10678/matrix-computations

[17] G. E. Blelloch, "Prefix sums and their applications," School of Computer Science, Carnegie Mellon University, Tech. Rep. CMU-CS-90-190, Nov. 1990. [Online]. Available: https://www.cs.cmu.edu/afs/cs.cmu.edu/project/scandal/public/papers/CMU-CS-90-190.html

[18] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013. [Online]. Available: https://www.informit.com/store/advanced-programming-in-the-unix-environment-9780321638007

[19] *Data Throughput, Latency & Handshaking*, Future Technology Devices International, Mar. 2024, AN232B-04 (v1.2). [Online]. Available: https://ftdichip.com/wp-content/uploads/2024/03/AN232B_04_DataThroughput_LatencyandHandshaking.pdf

[20] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications*

of the ACM, vol. 52, no. 4, pp. 65–76, 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[21] W. Simpson, "PPP in HDLC-like framing," RFC 1662, Internet Engineering Task Force, Jul. 1994. [Online]. Available: https://doi.org/10.17487/RFC1662

[22] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proceedings of the IRE*, vol. 49, no. 1, pp. 228–235, 1961. [Online]. Available: https://doi.org/10.1109/JRPROC.1961.287814

[23] P. Koopman and T. Chakravarty, "Cyclic redundancy code (CRC) polynomial selection for embedded networks," in *Proc. Int. Conf. Dependable Systems and Networks (DSN)*, 2004, pp. 145–154.

[24] A. D. Birrell and B. J. Nelson, "Implementing remote procedure calls," *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: https://doi.org/10.1145/2080.357392

[25] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, 1984. [Online]. Available: https://doi.org/10.1145/357401.357402

[26] *7 Series FPGAs Data Sheet: Overview*, AMD/Xilinx, 2020, DS180 (v2.6.1). [Online]. Available: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview

[27] R. Ginosar, "Metastability and synchronizers: A tutorial," *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011. [Online]. Available: https://doi.org/10.1109/MDT.2011.113

[28] *Vivado Design Suite User Guide: Using Constraints*, AMD/Xilinx, 2022, UG903 (v2022.2). [Online]. Available: https://docs.amd.com/r/2022.2-English/ug903-vivado-using-constraints

[29] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*. Springer, 2009. [Online]. Available: https://doi.org/10.1007/978-0-387-93820-2

[30] *UltraFast Design Methodology Guide for FPGAs and SoCs*, AMD/Xilinx, 2022, UG949 (v2022.2). [Online]. Available: https://docs.amd.com/r/2022.2-English/ug949-vivado-design-methodology

[31] *7 Series DSP48E1 Slice User Guide*, AMD/Xilinx, 2018, UG479 (v1.10). [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

[32] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, pp. 73:1–73:12. [Online]. Available: https://doi.org/10.1145/2807591.2807644

[33] T. Gingold, "GHDL: A VHDL 2008/93/87 simulator," accessed 2026-09-22. [Online]. Available: https://github.com/ghdl/ghdl

[34] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[35] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "Address-Sanitizer: A fast address sanity checker," in *USENIX Annual Technical Conference (ATC)*, 2012, pp. 309–318.

[36] C. Wolf, "Yosys open SYnthesis suite," accessed 2026-10-15. [Online]. Available: https://yosyshq.net/yosys/

[37] *IEEE Standard for Property Specification Language (PSL)*, IEEE, 2010, iEEE Std 1850-2010.

[38] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[39] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu, "Symbolic model checking without BDDs," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 1579. Springer, 1999, pp. 193–207. [Online]. Available: https://doi.org/10.1007/3-540-49059-0_14

[40] M. Sheeran, S. Singh, and G. Stålmarck, "Checking safety properties using induction and a SAT-solver," in *Formal Methods in Computer-Aided Design (FMCAD)*, ser. LNCS, vol. 1954. Springer, 2000, pp. 108–125. [Online]. Available: https://doi.org/10.1007/3-540-40922-X_8

[41] B. Dutertre, "Yices 2.2," in *Computer Aided Verification (CAV)*, ser. LNCS, vol. 8559. Springer, 2014, pp. 737–744. [Online]. Available: https://doi.org/10.1007/978-3-319-08867-9_49

[42] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978. [Online]. Available: https://doi.org/10.1109/C-M.1978.218136